Multivariate Distributions and Processes

Mark M. Fredrickson (mfredric@umich.edu)

Module 2: Generate

Computational Methods in Statistics and Data Science (DATASCI 406)

Joint Distributions

Factoring a joint distribution

The definition of a conditional density, $f(x_2 \mid x_1) = f(x_1, x_2)/f(x_1)$, rearranges to

$$f(x_1, x_2) = f(x_1)\, f(x_2 \mid x_1).$$

Applying it repeatedly **factors** any joint density into a product of conditionals:

$$f(x_1, \ldots, x_d) = f(x_1)\, f(x_2 \mid x_1)\, f(x_3 \mid x_1, x_2) \cdots f(x_d \mid x_1, \ldots, x_{d-1})$$

Any ordering of the coordinates works.

Sampling one coordinate at a time

The factorization gives a sampler:

1. Draw X_1 from $f(x_1)$.
2. Draw X_2 from $f(x_2 \mid X_1)$.
3. Continue until X_d is drawn from $f(x_d \mid X_1, \ldots, X_{d-1})$.

Each step is a **univariate** problem, so the methods from days 1–3 apply.

Day 2's mixtures were this with $d = 2$: draw the component or the parameter, then draw X given it.

Example: $f(x, y) = x + y$ on the unit square

The marginal of X, its CDF, and its quantile function:

$$f(x) = \int_0^1 (x + y)\, dy = x + \frac{1}{2}, \quad F(x) = \frac{x^2 + x}{2}, \quad Q_X(u) = \frac{-1 + \sqrt{1 + 8u}}{2}$$

The conditional of Y given $X = x$:

$$f(y \mid x) = \frac{x + y}{x + \frac{1}{2}}, \quad F(y \mid x) = \frac{xy + y^2/2}{x + \frac{1}{2}}, \quad Q(u \mid x) = -x + \sqrt{x^2 + u(2x + 1)}$$

Each quantile function comes from solving a quadratic for the root in $[0, 1]$.

In R

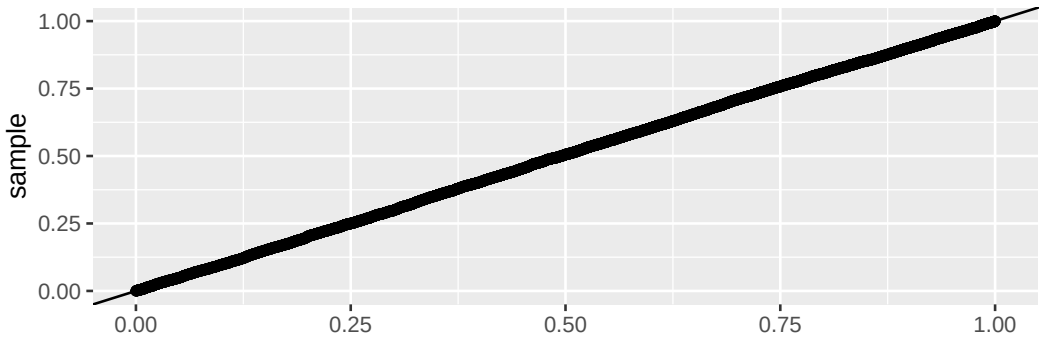
```r
qx <- function(p) (-1 + sqrt(1 + 8 * p)) / 2
qy_given_x <- function(p, x) -x + sqrt(x^2 + p * (2 * x + 1))

n <- 10000
x <- qx(runif(n))
y <- qy_given_x(runif(n), x) # y[i] is drawn given x[i]
```

Checking the marginal of Y

By symmetry Y has the same marginal as X, although we never drew Y from its marginal.

```
ggplot(data.frame(sample = y), aes(sample = sample)) +
  geom_qq(distribution = qx) +
  geom_abline(intercept = 0, slope = 1)
```



Checking the dependence

A QQ plot checks one coordinate at a time, so it would also pass if we drew Y independently of X. The correlation checks the dependence.

From f: $\mathrm{E}(XY) = 1/3$, $\mathrm{E}(X) = \mathrm{E}(Y) = 7/12$, and $\mathrm{Var}(X) = 11/144$, so

$$\mathrm{Cor}(X, Y) = \frac{1/3 - (7/12)^2}{11/144} = -\frac{1}{11}.$$

```
c(exact = -1/11, factored = cor(x, y),
  independent = cor(x, qx(runif(n))))
```

```
##       exact    factored independent
##    -0.09091    -0.09599     0.01192
```

Multivariate Normal Distributions

This section will use **vectors** and **matrices** in describing **multivariate distributions**.

A **vector** is a collection of values indexed $1, 2, \ldots, k$. E.g. X_3 is third element of $\boldsymbol{X}$.

A **matrix** is an $n \times m$ collection of values. X_{23} indicates the entry at the second row, third column of $\boldsymbol{X}$.

We can think of a matrix as m **vectors of length** n. Or a vector as a $n \times 1$ **matrix**.

The **transpose of a matrix** swaps the rows and columns. E.g., $X_{ij} = (\boldsymbol{X}^T)_{ji}$.

Matrix multiplication

We can **multiply matrices with conformable dimensions** such that if $\boldsymbol{A}$ is $n \times m$ and $\boldsymbol{B}$ is $m \times k$, we **can multiply** $\boldsymbol{AB}$ but **not** $\boldsymbol{BA}$.

Let $\boldsymbol{C} = \boldsymbol{AB}$, then

$$C_{ij} = \sum_{k=1}^{m} A_{ik} B_{kj} \quad \text{sum of pairwise products } i \text{ row of } \boldsymbol{A} \text{ with } j \text{ column of } \boldsymbol{B}$$

The **identity matrix** is a special $n \times n$ matrix such that

$$\boldsymbol{I}_n \boldsymbol{A} = \boldsymbol{A} \boldsymbol{I}_m = \boldsymbol{A}$$

A **matrix inverse** (if it exists) is such that for $\boldsymbol{A}$, $n \times n$:

$$\boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{I}_n$$

Multivariate Normal (MVN) Distributions

A special case of multivariate distributions with a very simple dependence structure is the multivariate distribution is the multivariate Normal distribution:

$$\boldsymbol{X} \sim N_d(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

- $\boldsymbol{\mu}$ is a vector of length d such that $\mathrm{E}(\boldsymbol{X}) = \boldsymbol{\mu}$ (the **mean**) – $\mathrm{E}(X_i) = \mu_i, i = 1, \ldots, d$
- $\boldsymbol{\Sigma}$ is the $d \times d$ **variance-covariance matrix** such that $\mathrm{Cov}(X_i, X_j) = \Sigma_{ij} = \Sigma_{ji}$
- When $\boldsymbol{\Sigma}$ is positive definite (usually), the PDF is:

$$f(\boldsymbol{x}) = |2\pi\boldsymbol{\Sigma}|^{-1/2} \exp\left\{-\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\boldsymbol{x} - \boldsymbol{\mu})\right\}$$

Motivation: many correlated tests

We test $m = 10$ outcomes measured on the same subjects. Under the null, the test statistics are $\boldsymbol{Z} \sim N_m(\mathbf{0}, \boldsymbol{\Sigma})$, with correlation induced by testing same subjects.

Bonferroni rejects H_{0i} when $|Z_i| > \Phi^{-1}(1 - \alpha/2m)$. Its family-wise error rate

$$\text{FWER} = P\left(\max_i |Z_i| > \Phi^{-1}(1 - \alpha/2m)\right)$$

is at most α for any $\boldsymbol{\Sigma}$. For independent tests it is $1 - (1 - \alpha/m)^m$.

How far below α is it when the tests are correlated? In general the FWER is a 10-dimensional normal integral. Monte Carlo can estimate it if we can draw from $N_m(\mathbf{0}, \boldsymbol{\Sigma})$.

Key intuition: what is the square root of Σ?

We know we can generate $X \sim N(\mu, \sigma^2)$ as

$$X = \sigma Z + \mu, \quad Z \sim N(0, 1)$$

For univariate Z, to get X we multiply Z by the square root of σ^2 (and add μ).

Think about the square root as the function that finds a such that $a \times a = b$:

$$\sqrt{b} = a : a \times a = b$$

We will do the same thing by finding the "square root" of the matrix $\mathbf{\Sigma}$: two matrices that multiply to $\mathbf{\Sigma} = \boldsymbol{C}\boldsymbol{C}^T$.

Generating MVN

Observe that we can generate the vector $\boldsymbol{Z}$ as a series of univariate, independent standard Normals:

$$\boldsymbol{Z} \sim N_d(\boldsymbol{0}, \boldsymbol{I}_d)$$

where I_d is the identity matrix:

$$\begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & \vdots \\ \vdots & 0 & \cdots & 1 \end{pmatrix}$$

If we can factor $\boldsymbol{\Sigma} = \boldsymbol{C}\boldsymbol{C}^T$, then by the linearity of multivariate Normals:

$$\boldsymbol{C}\boldsymbol{Z} + \boldsymbol{\mu} \sim N_d(\boldsymbol{\mu}, \boldsymbol{C}\boldsymbol{I}_d\boldsymbol{C}^T) = N_d(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

Pre-Proof

Some definitions. Let C_{ij} be the i row and j column of C. If we define $\boldsymbol{\Sigma} = \boldsymbol{C}\boldsymbol{C}^T$,

$$\begin{aligned} \mathrm{Var}(X_i) &= \Sigma_{ii} &&= \textstyle\sum_{k=1}^{d} C_{ik}^2 && \text{(sum of squared entries in } i\text{th row)} \\ \mathrm{Cov}(X_i, X_j) &= \Sigma_{ij} &&= \textstyle\sum_{k=1}^{d} C_{ik} C_{jk} && \text{(sum of pairwise products for } i \text{ and } j\text{)} \end{aligned}$$

Some facts:

- $Z_1, Z_2, \ldots Z_d \overset{\text{iid}}{\sim} N(0,1)$
- $Z_i^2 \sim \chi_1^2$, $\mathrm{E}(Z_i^2) = 1$
- For any RV Y, $\mathrm{Var}(cY) = c^2 \mathrm{Var}(Y)$
- For independent RVs, $\mathrm{Var}(Y_1 + Y_2) = \mathrm{Var}(Y_1) + \mathrm{Var}(Y_2)$ and $\mathrm{E}(Y_1 Y_2) = \mathrm{E}(Y_1)\,\mathrm{E}(Y_2)$

Proof: Marginal distribution

Claim: $\boldsymbol{X} = \boldsymbol{C}\boldsymbol{Z} \sim N_d(\mathbf{0}_d, \Sigma)$. First, show marginal distribution:

$$X_1 = C_{11}Z_1 + C_{12}Z_2 + \ldots C_{1d}Z_d \sim N(0, \sum_{k=1}^{d} C_{1k}^2) = N(0, \Sigma_{11})$$

$$X_2 = C_{21}Z_1 + C_{22}Z_2 + \ldots C_{2d}Z_d \sim N(0, \sum_{k=1}^{d} C_{2k}^2) = N(0, \Sigma_{22})$$

Proof: Covariance

We won't prove joint Normality, but we will show correct covariance:

$$\begin{aligned}
\mathrm{Cov}\left(X_1, X_2\right) &= \mathrm{E}\left(X_1 X_2\right) - \mathrm{E}\left(X_1\right)\mathrm{E}\left(X_2\right) \\
&= \mathrm{E}\left(\left(\sum_{k=1}^{d} C_{1k} Z_k\right)\left(\sum_{l=1}^{d} C_{2l} Z_l\right)\right) \\
&= \mathrm{E}\left(\sum_{k=1}^{d}\sum_{l=1}^{d} C_{1k} C_{2l} Z_k Z_l\right) \\
&= \sum_{k=1}^{d}\sum_{l=1}^{d} C_{1k} C_{2l} \mathrm{E}\left(Z_k Z_l\right) \\
&= \sum_{k=1}^{d} C_{1k} C_{2k} \mathrm{E}\left(Z_k^2\right) + \sum_{k \neq l} C_{1k} C_{2l} \mathrm{E}\left(Z_k Z_l\right) = \sum_{k=1}^{d} C_{1k} C_{2k} = \Sigma_{12}
\end{aligned}$$

Factoring Σ

Any positive definite $\boldsymbol{\Sigma}$, which includes most covariance matrices, has a **Cholesky factorization**

$$\boldsymbol{\Sigma} = \boldsymbol{C}\boldsymbol{C}^T, \quad \boldsymbol{C} \text{ lower triangular.}$$

In R, `chol(Sigma)` returns the upper triangular $\boldsymbol{C}^T$, so use `C <- t(chol(Sigma))`.

The Cholesky decomposition as factoring

For two coordinates with unit variances and correlation ρ:

$$\boldsymbol{C} = \begin{pmatrix} 1 & 0 \\ \rho & \sqrt{1-\rho^2} \end{pmatrix}, \qquad \boldsymbol{X} = \boldsymbol{C}\boldsymbol{Z}: \quad X_1 = Z_1, \quad X_2 = \rho X_1 + \sqrt{1-\rho^2}\, Z_2$$

- X_1 is drawn from its marginal, $N(0, 1)$.
- X_2 is drawn from its conditional given X_1, $N(\rho X_1, 1 - \rho^2)$.

Each row of a lower triangular $\boldsymbol{C}$ adds one new Z_i, so in general X_i is drawn given $X_1, \ldots, X_{i-1}$. In other words, using the factoring we started with.

Back to the tests

```r
m <- 10
bonferroni <- qnorm(1 - 0.05 / (2 * m))

max_abs_z <- function(rho, B = 100000) {
  Sigma <- matrix(rho, m, m)
  diag(Sigma) <- 1
  C <- t(chol(Sigma))
  Z <- C %*% matrix(rnorm(m * B), m) # each column is a draw
  apply(abs(Z), 2, max)
}
```

Bonferroni with correlated tests

```
results <- tibble(rho = c(0, 0.3, 0.6, 0.9)) %>%
  mutate(maxes = map(rho, max_abs_z),
         fwer = map_dbl(maxes, ~ mean(.x > bonferroni)),
         cutoff = map_dbl(maxes, ~ quantile(.x, 0.95)))
select(results, -maxes)
```

```
## # A tibble: 4 x 3
##     rho   fwer cutoff
##   <dbl>  <dbl>  <dbl>
## 1   0   0.0488   2.80
## 2   0.3 0.0467   2.78
## 3   0.6 0.0345   2.67
## 4   0.9 0.0167   2.38
```

What the simulation shows

At $\rho = 0$ the exact FWER is $1 - 0.995^{10} = 0.0489$, which checks the sampler.

As ρ grows, Bonferroni's FWER falls well below 5%, so it rejects less often than it could.

Rejecting when $|Z_i|$ exceeds `cutoff`, the 95th percentile of $\max_i |Z_i|$, gives a FWER of 5% (Bonferroni uses 2.81).

Data: blue crabs

`MASS::crabs` has five size measurements, in mm, of 50 male and 50 female blue crabs: frontal lobe (FL), rear width (RW), carapace length (CL) and width (CW), and body depth (BD). Do the sexes differ? That is five tests.

```
blue <- subset(MASS::crabs, sp == "B")
vars <- c("FL", "RW", "CL", "CW", "BD")
male <- as.matrix(blue[blue$sex == "M", vars])
female <- as.matrix(blue[blue$sex == "F", vars])
z <- (colMeans(male) - colMeans(female)) /
  sqrt(apply(male, 2, var) / 50 + apply(female, 2, var) / 50)
round(z, 2)
```

```
##    FL    RW    CL    CW    BD
##  2.68 -0.92  2.94  2.75  2.57
```

The null distribution for the crabs

Under the null, $\boldsymbol{Z} \approx N_5(\mathbf{0}, \boldsymbol{R})$, where $\boldsymbol{R}$ is the correlation of the measurements within a sex. Every correlation is above 0.95.

```
# center each sex at its own mean, then correlate
R <- cor(rbind(scale(male, scale = FALSE), scale(female, scale = FALSE)))
C <- t(chol(R))
maxes <- apply(abs(C %*% matrix(rnorm(5 * 100000), 5)), 2, max)
quantile(maxes, 0.95)
```

```
##    95%
## 2.098
```

Bonferroni's cutoff for five tests is 2.58.

Adjusted p-values

Each method gives an adjusted p-value, and we reject when it is below 0.05:

- Bonferroni: $\min(1, m\,p_i)$
- Max-$|Z|$: $P(\max_j |Z_j| \geq |z_i|)$, estimated by the fraction of `maxes` at least $|z_i|$

```
p <- 2 * pnorm(-abs(z))
max_z <- sapply(abs(z), function(t) mean(maxes >= t))
round(cbind(z, p, bonferroni = pmin(1, 5 * p), max_z), 3)
```

```
##          z     p bonferroni max_z
## FL   2.683 0.007      0.036 0.011
## RW -0.921 0.357      1.000 0.437
## CL   2.941 0.003      0.016 0.005
## CW   2.747 0.006      0.030 0.009
## BD   2.570 0.010      0.051 0.015
```

What the crabs show

Body depth (BD) is significant using the max-$|Z|$ cutoff but not using Bonferroni.

Every max-$|Z|$ p-value is about a third of the Bonferroni one. With correlations near 1, the five measurements nearly measure one thing, so the max-$|Z|$ p-values are close to the unadjusted ones.

MVN Summary

- General dependence is **difficult to categorize**.
- The Normal distribution equates **dependence to covariance/correlation** (defining characteristic of the distribution).
- Also serves a **common approximation** (via CLT)
- Extend the idea of $Z \sim N(0,1) \Rightarrow X = \sigma Z + \mu \sim N(\mu, \sigma^2)$, to

$$\boldsymbol{Z} \sim N_d(\boldsymbol{0}, \boldsymbol{I}_d) \Rightarrow \boldsymbol{C}\boldsymbol{Z} + \boldsymbol{\mu} \sim N_d(\boldsymbol{\mu}, \boldsymbol{C}\boldsymbol{C}^T)$$

- Need a concept the **square root of a matrix** (e.g., Cholesky factorization)

Stochastic Processes

Stochastic Processes

A **stochastic process** is a collection of random variables indexed by time or space:

$$\{X(t) : t \in \mathcal{T}\}$$

The index set $\mathcal{T}$ can be discrete, such as $\{0, 1, 2, \ldots\}$, or continuous, such as $[0, \infty)$.

To simulate a process we draw $X(t)$ at finitely many times. That is a random vector, so we factor its joint distribution and draw one time at a time.

Example: Random Walk

- Start from $X(0) = x_0$.
- For $t = 1, 2, \ldots$, set

$$X(t) = X(t-1) + S(t)$$

where $P(S(t) = 1) = P(S(t) = -1) = 0.5$ and the steps $S(t)$ are independent.

Unrolling the recursion,

$$X(t) = x_0 + \sum_{i=1}^{t} S(i).$$

The random walk is a factorization

Factor the walk's joint distribution over time:

$$f(x_1, \ldots, x_k \mid x_0) = f(x_1 \mid x_0)\, f(x_2 \mid x_1, x_0) \cdots f(x_k \mid x_{k-1}, \ldots, x_0)$$

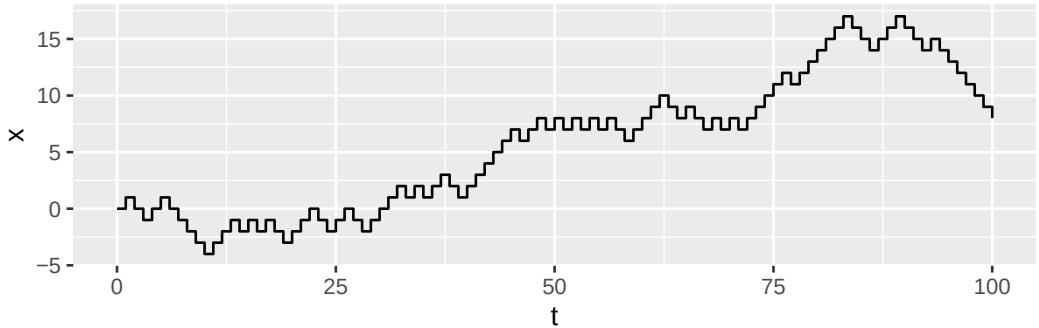
Each factor depends only on the previous value:

$$f(x_t \mid x_{t-1}, \ldots, x_0) = f(x_t \mid x_{t-1}), \qquad X(t) \mid X(t-1) = x \text{ is } x \pm 1 \text{ with probability } 1/2$$

A process with dependence limited previous steps is a **Markov chain**.

Implementation

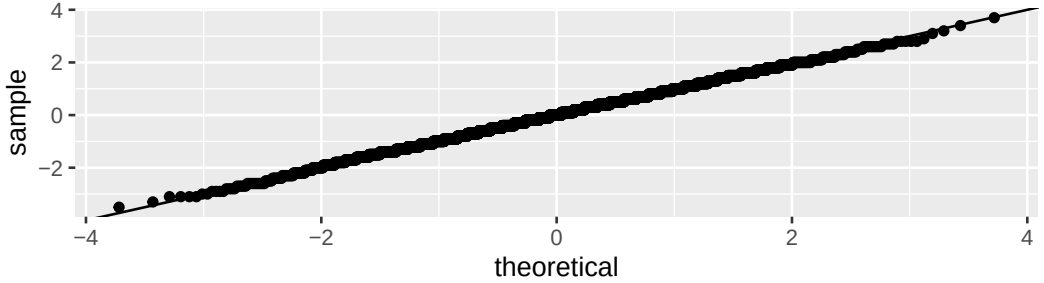
```
s <- sample(c(1, -1), 100, replace = TRUE)
x <- c(0, cumsum(s))
ggplot(tibble(t = 0:100, x), aes(t, x)) + geom_step()
```



Many walks

$X(t)$ is a sum of t independent steps with variance 1, so $\mathrm{Var}(X(t)) = t$, and by the CLT $X(t)/\sqrt{t} \approx N(0,1)$.

```
walk_end <- function(k) sum(sample(c(1, -1), k, replace = TRUE))
ends <- replicate(5000, walk_end(400))
ggplot(tibble(sample = ends / sqrt(400)), aes(sample = sample)) +
  geom_qq() + geom_abline(intercept = 0, slope = 1)
```



Brownian motion

Shrink the steps: take a step every Δt, with variance Δt. As $\Delta t \to 0$ the walk converges to **Brownian motion** $W(t)$, $t \geq 0$:

- $W(0) = 0$
- $W(t) - W(s) \sim N(0, t - s)$ for $s < t$
- Increments over non-overlapping intervals are independent.

So $W(t) \sim N(0, t)$, and the paths are continuous but nowhere smooth.

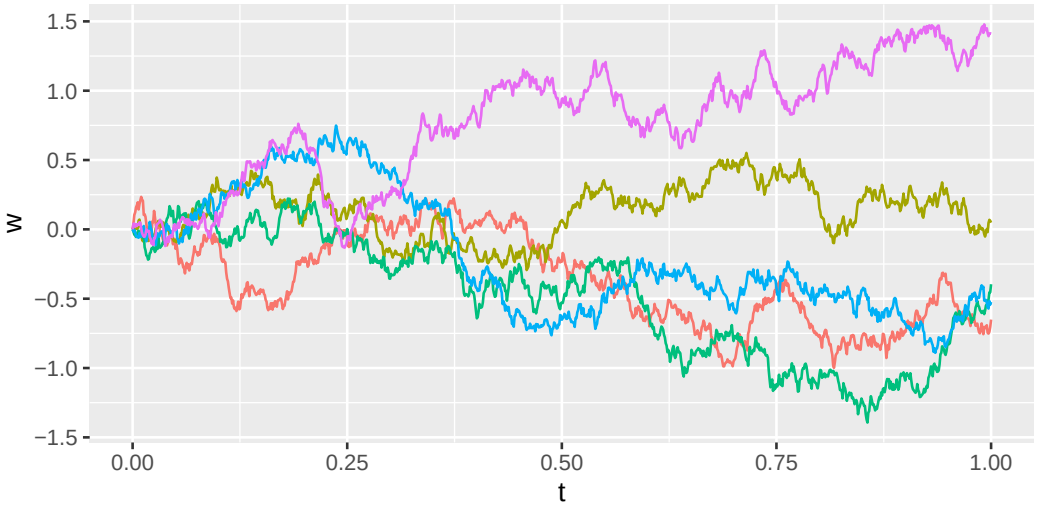
Simulating Brownian motion

We can only draw W on a grid $0 = t_0 < t_1 < \cdots < t_n$. Factoring as for the walk,

$$W(t_i) \mid W(t_{i-1}) \sim N\big(W(t_{i-1}),\, t_i - t_{i-1}\big).$$

```
brownian <- function(n, t_max = 1) {
  dt <- t_max / n
  tibble(t = seq(0, t_max, length.out = n + 1),
         w = c(0, cumsum(rnorm(n, sd = sqrt(dt)))))
}
```

Five paths



Checking the sampler: the maximum

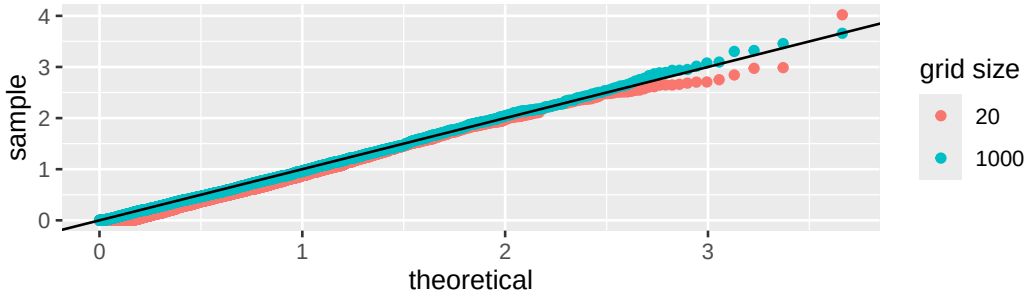
The maximum $M = \max_{0 \le t \le 1} W(t)$ has the same distribution as $|Z|$ for $Z \sim N(0,1)$ (the **reflection principle**), so

$$Q_M(p) = \Phi^{-1}\left(\frac{1+p}{2}\right).$$

```
q_max <- function(p) qnorm((1 + p) / 2)
max_w <- function(n) max(brownian(n)$w)
sim_max <- tibble(n = rep(c(20, 1000), each = 2000)) %>%
  mutate(m = map_dbl(n, max_w))
```

A coarse grid misses the peaks

```
ggplot(sim_max, aes(sample = m, color = factor(n))) +
  geom_qq(distribution = q_max) +
  geom_abline(intercept = 0, slope = 1) + labs(color = "grid size")
```



With 20 grid points the maximum is too small, because the peaks fall between grid points.

Brownian bridge

A **Brownian bridge** is Brownian motion on $[0, 1]$ conditioned to return to 0: $B(t) = W(t) \mid W(1) = 0$.

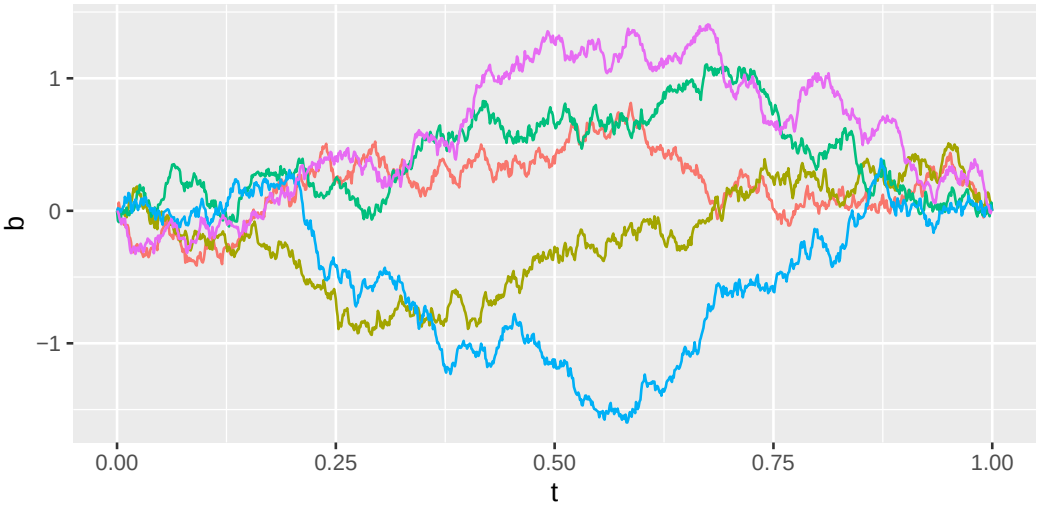
Any ordering of the coordinates works, so draw the endpoint first and the middle given the ends:

$$W(t) \mid W(1) = w_1 \sim N\big(t\, w_1,\ t(1-t)\big)$$

Subtracting the straight line from 0 to $W(1)$ removes the endpoint, so $B(t) = W(t) - t\, W(1)$.

```
bridge <- function(n) {
  mutate(brownian(n), b = w - t * last(w))
}
```

Five bridges

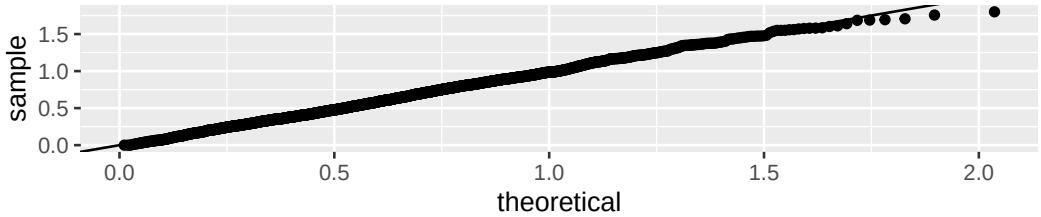


Checking the bridge

The maximum $\max_t B(t)$ has $P(\max_t B(t) > a) = e^{-2a^2}$, so

$$Q(p) = \sqrt{-\log(1-p)/2}.$$

```
q_bridge_max <- function(p) sqrt(-log(1 - p) / 2)
bridge_max <- replicate(2000, max(bridge(1000)$b))
ggplot(tibble(sample = bridge_max), aes(sample = sample)) +
  geom_qq(distribution = q_bridge_max) +
  geom_abline(intercept = 0, slope = 1)
```



Poisson process

Events arrive at times $T_1 < T_2 < \cdots$, and $N(t)$ counts the arrivals up to time t.

- The gaps $T_k - T_{k-1}$ are independent $\text{Exp}(\lambda)$, drawn by inversion as on day 1.
- Each arrival time is a sum of gaps, $T_k = E_1 + \cdots + E_k$, as on day 2.
- Then $N(t) \sim \text{Poisson}(\lambda t)$.

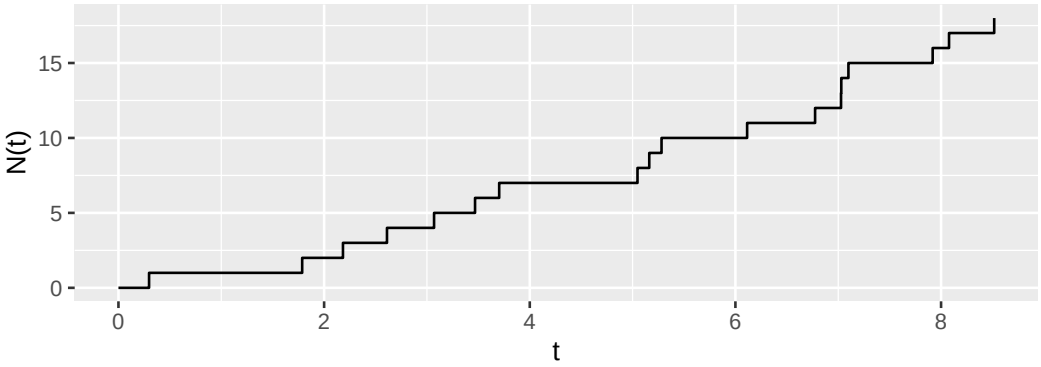
Factoring again: $T_k \mid T_{k-1} = s$ is s plus an $\text{Exp}(\lambda)$ gap.

Implementation

```
arrivals <- function(lambda, t_max) {
  times <- c()
  t <- rexp(1, lambda)
  while (t <= t_max) {
    times <- c(times, t)
    t <- t + rexp(1, lambda)
  }
  times
}
```

One path of $N(t)$

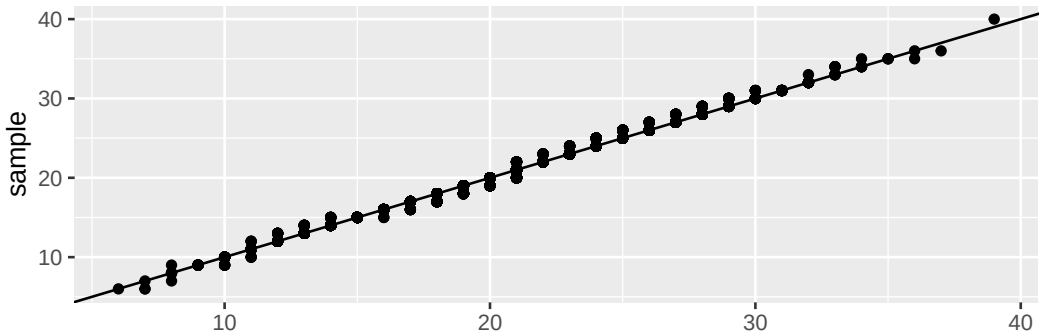
```
ggplot(tibble(t = c(0, arrivals(2, 10))), aes(t, seq_along(t) - 1)) +
  geom_step() + labs(y = "N(t)")
```



Checking the count

With $\lambda = 2$ and $t = 10$, $N(10) \sim \text{Poisson}(20)$.

```
counts <- replicate(5000, length(arrivals(2, 10)))
ggplot(tibble(sample = counts), aes(sample = sample)) +
  geom_qq(distribution = qpois, dparams = list(lambda = 20)) +
  geom_abline(intercept = 0, slope = 1)
```



Some other stochastic processes

- **Martingale**: $\mathrm{E}\left(X(t) \mid X(t-1), X(t-2), \ldots\right) = X(t-1)$. The symmetric random walk is one.
- **Spatial Poisson process**: in a region of area A, draw $N \sim \text{Poisson}(\lambda A)$ and place N points uniformly at random.
- **Autoregressive processes**: $X(t) = \phi X(t-1) + \epsilon(t)$, which reduces to the random walk at $\phi = 1$.

Stochastic process summary

- A process is a collection of dependent random variables $\{X(t)\}$, indexed by time or space.
- We simulate one at a time, drawing each $X(t)$ given the earlier ones. This is the factorization from the start of the deck.
- The random walk, Brownian motion and the Poisson process depend on the past only through the most recent value.
- A continuous-time process has to be drawn on a grid, and a coarse grid can bias what we estimate.